

Manual de Código

Usuarios

ATRIBUTO	TIPO DE DATO SQL	RESTRICCIÓN	DESCRIPCIÓN
ID	BIGINT	PK, Auto-incremental	Identificador único de usuario
NOMBRE	VARCHAR(255)	NOT NULL	Nombre completo o alias del usuario
EMAIL	VARCHAR(255)	NOT NULL	Correo electrónico (credencial de acceso)
PASSWORD	VARCHAR(255)	NOT NULL	Contraseña encriptada por seguridad
FECHA_REGISTRO	DATE TIME	NOT NULL	Fecha y hora en la que se creó el registro

Medidas

ATRIBUTO	TIPO DE DATO SQL	RESTRICCIÓN	DESCRIPCIÓN
ID	BIGINT	PK, Auto-incremental	Identificador único de medidas
PESO	DOUBLE	NOT NULL	Peso del usuario en kilogramos

PORCENTAJE_GRASA	DOUBLE	Opcinal	Índice de grasa corporal del usuario
FECHA	DATE TIME	NOT NULL	Fecha exacta a de la toma de medidas
USUARIO_ID	BIGINT	FK	Referencia de usuario al que pertenece la medida

Objetivos

ATRIBUTO	TIPO DE DATO SQL	RESTRICCIÓN	DESCRIPCIÓN
ID	BIGINT	PK, Auto_increment	Identificador único de objetivo
TIPO	VARCHAR(255)	NOT NULL	Tipo de objetivo (ej, Peso, Grasa, Fuerza...)
VALOR_OBJETIVO	DOUBLE	NOT NULL	Valor numérico a alcanzar
FECHA_LIMITE	DATE	NULL	Fecha límite estimada para cumplir el objetivo
COMPLETADO	BIT(1)	NOT NULL	Indica si el objetivo se ha cumplido (0 o 1)
USUARIO_ID	BIGINT	FK, NULL	Referencia al usuario (usuarios.id)

Equipamiento

ATRIBUTO	TIPO DE DATO SQL	RESTRICCIÓN	DESCRIPCIÓN
ID	BIGINT	PK, Auto-incremental	Identificador único de equipamiento
NOMBRE	VARCHAR(255)	UNIQUE, NOT NULL	Nombre del equipamiento

Ejercicios

ATRIBUTO	TIPO DE DATO SQL	RESTRICCIÓN	DESCRIPCIÓN
ID	BIGINT	PK, Auto-incremental	Identificador único de usuario
NOMBRE	VARCHAR(255)	NOT NULL	Nombre del movimiento
GRUPO_MUSCULAR	VARCHAR(255)	NOT NULL	Zona anatómica principal implicada

Ejercicio_equipamiento

ATRIBUTO	TIPO DE DATO SQL	RESTRICCIÓN	DESCRIPCIÓN
EJERCICIO_ID	BIGINT	PK*, FK, NOT NULL	Referencia al ejercicio (ejercicios.id)
EQUIPAMIENTO_ID	BIGINT	PK*,FK, NOT NULL	Referencia al equipo

compatible
(equipamiento.id)

Rutinas

ATRIBUTO	TIPO DE DATO SQL	RESTRICCIÓN	DESCRIPCIÓN
ID	BIGINT	PK, Auto-incremental	Identificador único de usuario
NOMBRE	VARCHAR(255)	NOT NULL	Nombre descriptivo

Rutina ejercicio

ATRIBUTO	TIPO DE DATO SQL	RESTRICCIÓN	DESCRIPCIÓN
RUTINA_ID	BIGINT	PK,FK	Referencia a la tabla rutinas
EJERCICIO_ID	BIGINT	PK, FK	Referencia a la tabla ejercicios

Entrenamientos

ATRIBUTO	TIPO DE DATO SQL	RESTRICCIÓN	DESCRIPCIÓN
ID	BIGINT	PK, Auto-incremental	Identificador único de usuario
FECHA	DATETIME	NOT NULL	Fecha y hora de realización

DURACIÓN_MINUTOS	INT	NOT NULL	Tiempo total de duración de la sesión
USUARIO_ID	BIGINT	FK	Referencia al usuario que entrenó
RUTINA_ID	BIGINT	FK, Opcional	Referencia a la rutina base seguida (si la hay)

Ejercicio entrenamiento

ATRIBUTO	TIPO DE DATO SQL	RESTRICCIÓN	DESCRIPCIÓN
ID	BIGINT	PK, Auto-incremental	Identificador del bloque de ejercicio en la sesión.
ORDEN	INT(11)	NULL	Orden de la ejecución dentro de la sesión
NOTAS	VARCHAR(255)	NULL	Observaciones específicas para esta ejecución
EJERCICIO_ID	BIGINT	FK, NLL	Referencia al entrenamiento (ejercicios.id)
ENTRENAMIENTO_ID	BIGINT	FK, NULL	Referencia al entrenamiento (entrenamiento.id)

EQUIPAMIENTO_ID	BIGINT	FK, NULL	Equipo particular usado en la sesión (equipamiento.id)
-----------------	--------	----------	--

Series

ATRIBUTO	TIPO DE DATO SQL	RESTRICCIÓN	DESCRIPCIÓN
ID	BIGINT	PK, Auto-incremental	Identificador único de la serie realizada
TIPO	VARCHAR(255)	NULL	Tipo de serie (ej. Efectiva, Calentamiento)
REPETICIONES	INT	NOT NULL	Número de alzamientos completados
PESO	DOUBLE	NOT NULL	Carga externa levantada en kilogramos
RPE	INT	NOT NULL	Escala de esfuerzo percibido (1-10)
EJERCICIO_ENTRENAMIENTO_ID	BIGINT	FK, NULL	Vincula la serie con el bloque del ejercicio (ejercicio_entrenamiento.id)

Relaciones entre tablas

Medida

- ManyToOne con Usuario

Objetivo

- ManyToOne con Usuario

Rutina

- ManyToMany con Ejercicio

Serie

- ManyToOne con EjercicioEntrenamiento

Usuario

- *(No declara mapeos inversos explícitos en esta clase, actúa como entidad padre en la BD)*

Ejercicio

- ManyToMany con Equipamiento

EjercicioEntrenamiento

- ManyToOne con Entrenamiento
- ManyToOne con Ejercicio
- ManyToOne con Equipamiento
- OneToMany con Serie

Entrenamiento

- ManyToOne con Usuario
- ManyToOne con Rutina
- OneToMany con EjercicioEntrenamiento

Equipamiento

- ManyToMany con Ejercicio

Listado de Clases

Listado de clases

Clase: Medida Propiedades:

- id (Long, privado)
- fecha (LocalDate, privado)

- pesoCorporal (Double, privado)
- porcentajeGrasa (Double, privado)
- usuario (Usuario, privado)

Métodos:

- getId(): Long
- setId(Long id): void
- getFecha(): LocalDate
- setFecha(LocalDate fecha): void
- getPesoCorporal(): Double
- setPesoCorporal(Double pesoCorporal): void
- getPorcentajeGrasa(): Double
- setPorcentajeGrasa(Double porcentajeGrasa): void
- getUsuario(): Usuario
- setUsuario(Usuario usuario): void

Clase: Objetivo Propiedades:

- id (Long, privado)
- tipo (String, privado)
- valorObjetivo (Double, privado)
- fechaLimite (LocalDate, privado)
- completado (boolean, privado)
- usuario (Usuario, privado)

Métodos:

- getId(): Long
- setId(Long id): void
- getTipo(): String
- setTipo(String tipo): void
- getValorObjetivo(): Double
- setValorObjetivo(Double valorObjetivo): void

- getFechaLimite(): LocalDate
- setFechaLimite(LocalDate fechaLimite): void
- isCompletado(): boolean
- setCompletado(boolean completado): void
- getUsuario(): Usuario
- setUsuario(Usuario usuario): void

Clase: Rutina Propiedades:

- id (Long, privado)
- nombre (String, privado)
- tipo (String, privado)
- ejercicios (List, privado)

Métodos:

- getId(): Long
- setId(Long id): void
- getNombre(): String
- setNombre(String nombre): void
- getTipo(): String
- setTipo(String tipo): void
- getEjercicios(): List
- setEjercicios(List ejercicios): void

Clase: Serie Propiedades:

- id (Long, privado)
- peso (Double, privado)
- repeticiones (Integer, privado)
- rpe (Integer, privado)
- ejercicioEntrenamiento (EjercicioEntrenamiento, privado)
- tipo (String, privado)

Métodos:

- getId(): Long
- setId(Long id): void
- getPeso(): Double
- setPeso(Double peso): void
- getRepeticiones(): Integer
- setRepeticiones(Integer repeticiones): void
- getRpe(): Integer
- setRpe(Integer rpe): void
- getEjercicioEntrenamiento(): EjercicioEntrenamiento
- setEjercicioEntrenamiento(EjercicioEntrenamiento ejercicioEntrenamiento): void
- getTipo(): String
- setTipo(String tipo): void

Clase: Usuario Propiedades:

- id (Long, privado)
- email (String, privado)
- password (String, privado)
- nombre (String, privado)
- fechaRegistro (LocalDate, privado)

Métodos:

- getId(): Long
- setId(Long id): void
- getEmail(): String
- setEmail(String email): void
- getPassword(): String
- setPassword(String password): void
- getNombre(): String
- setNombre(String nombre): void

- getFechaRegistro(): LocalDate
- setFechaRegistro(LocalDate fechaRegistro): void

Clase: Ejercicio Propiedades:

- id (Long, privado)
- nombre (String, privado)
- grupoMuscular (String, privado)
- descripcion (String, privado)
- equiposDisponibles (List, privado)

Métodos:

- getId(): Long
- setId(Long id): void
- getNombre(): String
- setNombre(String nombre): void
- getGrupoMuscular(): String
- setGrupoMuscular(String grupoMuscular): void
- getDescripcion(): String
- setDescripcion(String descripcion): void
- getEquiposDisponibles(): List
- setEquiposDisponibles(List equiposDisponibles): void

Clase: EjercicioEntrenamiento Propiedades:

- id (Long, privado)
- entrenamiento (Entrenamiento, privado)
- ejercicio (Ejercicio, privado)
- equipamiento (Equipamiento, privado)
- orden (Integer, privado)
- notas (String, privado)
- series (List, privado)

Métodos:

- getId(): Long
- setId(Long id): void
- getEntrenamiento(): Entrenamiento
- setEntrenamiento(Entrenamiento entrenamiento): void
- getEjercicio(): Ejercicio
- setEjercicio(Ejercicio ejercicio): void
- getEquipamiento(): Equipamiento
- setEquipamiento(Equipamiento equipamiento): void
- getOrden(): Integer
- setOrden(Integer orden): void
- getNotas(): String
- setNotas(String notas): void
- getSeries(): List
- setSeries(List series): void

Clase: Entrenamiento Propiedades:

- id (Long, privado)
- fecha (LocalDateTime, privado)
- duracionMinutos (Integer, privado)
- usuario (Usuario, privado)
- rutina (Rutina, privado)
- ejerciciosEntrenamiento (List, privado)

Métodos:

- getId(): Long
- setId(Long id): void
- getFecha(): LocalDateTime
- setFecha(LocalDateTime fecha): void
- getDuracionMinutos(): Integer
- setDuracionMinutos(Integer duracionMinutos): void

- `getUsuario(): Usuario`
- `setUsuario(Usuario usuario): void`
- `getRutina(): Rutina`
- `setRutina(Rutina rutina): void`
- `getEjerciciosEntrenamiento(): List`
- `setEjerciciosEntrenamiento(List ejerciciosEntrenamiento): void`

Clase: Equipamiento Propiedades:

- `id (Long, privado)`
- `nombre (String, privado)`
- `ejercicios (List, privado)`

Métodos:

- `getId(): Long`
- `setId(Long id): void`
- `getNombre(): String`
- `setNombre(String nombre): void`
- `getEjercicios(): List`
- `setEjercicios(List ejercicios): void`

Listado de Datos

Clase: EquipamientoController Propiedades privadas:

- `equipamientoRepository`: Instancia del repositorio de equipamiento.

Métodos Públicos:

- `findAll(): List`
 - **Descripción:** Muestra y devuelve todos los equipos de la base de datos, mapeándolos a un DTO limpio para evitar bucles de dependencias con la entidad Ejercicio.
 - **Ruta:** `/api/equipamiento` (método GET)
 - **Respuesta:** Devuelve un objeto JSON (lista) con todos los equipamientos.

Clase: MedidaController Propiedades privadas:

- medidaRepository: Instancia del repositorio de medidas.
- usuarioRepository: Instancia del repositorio de usuarios.

Métodos Públicos:

- registrarMedida(MedidaRequest request): ResponseEntity<?>
 - **Descripción:** Registra una nueva medición corporal de un usuario. Valida la existencia del usuario y establece automáticamente la fecha actual desde el servidor para garantizar la consistencia.
 - **Ruta:** /api/medidas (método POST)
 - **Parámetros:** * request: Objeto DTO que contiene los datos de la medida (peso, porcentaje de grasa y ID del usuario).
 - **Respuesta:** Devuelve la medida guardada (código 200 OK) o un error (código 400 Bad Request) si el usuario no existe.
- obtenerHistorial(Long usuarioid): ResponseEntity<List>
 - **Descripción:** Consulta el historial evolutivo de medidas de un usuario en concreto, ordenadas cronológicamente de forma descendente.
 - **Ruta:** /api/medidas/usuario/{usuarioid} (método GET)
 - **Parámetros:** * usuarioid: Identificador único del usuario (extraído de la URL).
 - **Respuesta:** Devuelve una lista JSON de medidas con código 200 OK (o un array vacío si no hay registros).

Clase: ObjetivoController Propiedades privadas:

- objetivoRepository: Instancia del repositorio de objetivos.
- usuarioRepository: Instancia del repositorio de usuarios.

Métodos Públicos:

- crearObjetivo(ObjetivoRequest request): ResponseEntity<?>
 - **Descripción:** Crea y registra una nueva meta u objetivo físico para un usuario validando primero su existencia.
 - **Ruta:** /api/objetivos (método POST)

- **Parámetros:** * request: Objeto DTO con los datos del objetivo (usuariold, tipo, valorObjetivo, fechaLimite).
- **Respuesta:** Devuelve el objetivo guardado (200 OK) o un error si el usuario no existe (400 Bad Request).
- obtenerPorUsuario(Long usuariold): List
 - **Descripción:** Obtiene la lista de objetivos activos asociados a un usuario específico.
 - **Ruta:** /api/objetivos/usuario/{usuariold} (método GET)
 - **Parámetros:** * usuariold: Identificador del usuario (extraído de la URL).
 - **Respuesta:** Devuelve la lista de objetivos serializada a JSON.
- actualizarEstado(Long id, boolean completado): ResponseEntity<?>
 - **Descripción:** Actualiza únicamente el estado de completado de un objetivo existente.
 - **Ruta:** /api/objetivos/{id}/estado (método PUT)
 - **Parámetros:** * id: Identificador del objetivo.
 - completado: Variable booleana con el nuevo estado a aplicar.
 - **Respuesta:** Devuelve el objetivo actualizado (200 OK) o un estado 404 (Not Found) si no existe.

Clase: RutinaController Propiedades privadas:

- rutinaRepository: Instancia del repositorio de rutinas.
- ejercicioRepository: Instancia del repositorio de ejercicios.

Métodos Públicos:

- crearRutina(RutinaRequest request): ResponseEntity<?>
 - **Descripción:** Crea una nueva rutina vinculando una lista de ejercicios existentes en el catálogo.
 - **Ruta:** /api/rutinas (método POST)
 - **Parámetros:** * request: DTO con el nombre, tipo y lista de IDs de los ejercicios.

- **Respuesta:** Devuelve el DTO limpio de la rutina guardada (200 OK) o un error si la lista de ejercicios está vacía (400 Bad Request).
- obtenerTodas(): ResponseEntity<List>
 - **Descripción:** Obtiene todas las rutinas registradas en el sistema.
 - **Ruta:** /api/rutinas (método GET)
 - **Respuesta:** Devuelve una lista JSON de rutinas mapeadas a objetos de respuesta (200 OK).

Métodos Privados:

- mapearARutinaResponse(Rutina rutina)
 - **Descripción:** Mapea la entidad Rutina al DTO RutinaResponse para evitar dependencias cíclicas en la respuesta.
 - **Parámetros:** * rutina: Objeto de tipo Rutina.
 - **Retorno:** Devuelve el objeto RutinaResponse debidamente formateado.

Clase: UsuarioController Propiedades privadas:

- usuarioRepository: Instancia del repositorio de usuarios.
- jwtUtil: Componente utilitario para la generación y validación de tokens JWT.

Métodos Públicos:

- crearUsuario(Usuario usuario): Usuario
 - **Descripción:** Registra un nuevo usuario en el sistema. Intercepta la contraseña en texto plano y la encripta utilizando BCrypt antes de persistirla.
 - **Ruta:** /api/usuarios (método POST)
 - **Parámetros:** * usuario: Objeto Usuario extraído del cuerpo de la petición.
 - **Respuesta:** Devuelve el usuario persistido en la base de datos (con su ID generado).
- login(LoginRequest request): ResponseEntity<?>
 - **Descripción:** Autentica a un usuario existente comprobando su email y el hash de su contraseña. Si es correcto, emite un token JWT.

- **Ruta:** /api/usuarios/login (método POST)
- **Parámetros:** * request: Objeto DTO que contiene el email y la contraseña.
- **Respuesta:** Devuelve el token JWT y los datos del usuario (200 OK) o un error genérico (401 Unauthorized) si fallan las credenciales.

Clase: EjercicioController Propiedades privadas:

- ejercicioRepository: Instancia del repositorio de ejercicios.

Métodos Públicos:

- findAll(): List
 - **Descripción:** Obtiene el catálogo completo de ejercicios disponibles en el gimnasio.
 - **Ruta:** /api/ejercicios (método GET)
 - **Respuesta:** Devuelve una lista JSON de ejercicios mapeados como DTOs.
- crearEjercicio(Ejercicio ejercicio): EjercicioRequest
 - **Descripción:** Registra un nuevo ejercicio en el catálogo de la aplicación.
 - **Ruta:** /api/ejercicios (método POST)
 - **Parámetros:** * ejercicio: Entidad recibida en el cuerpo de la petición.
 - **Respuesta:** Devuelve un DTO representando el ejercicio guardado.

Métodos Privados:

- convertirAEjercicioDTO(Ejercicio entidad)
 - **Descripción:** Convierte una entidad Ejercicio a su representación de transferencia DTO.
 - **Parámetros:** * entidad: Objeto de tipo Ejercicio.
 - **Retorno:** Devuelve el EjercicioRequest formateado.
- convertirAEquipamientoDTO(Equipamiento equipo)
 - **Descripción:** Convierte una entidad Equipamiento a su representación DTO.
 - **Parámetros:** * equipo: Objeto de tipo Equipamiento.

- **Retorno:** Devuelve el EquipamientoRequest formateado.

Clase: EntrenamientoController Propiedades privadas:

- entrenamientoService: Servicio que procesa la lógica de negocio de los entrenamientos.
- entrenamientoRepository: Instancia del repositorio de entrenamientos.

Métodos Públicos:

- registrarEntrenamiento(EntrenamientoRequest request): ResponseEntity<?>
 - **Descripción:** Registra un nuevo entrenamiento completo con su respectiva estructura de ejercicios y series (delegando la lógica al servicio).
 - **Ruta:** /api/entrenamientos (método POST)
 - **Parámetros:** * request: DTO validado con los datos del entrenamiento.
 - **Respuesta:** Devuelve el DTO del entrenamiento persistido (200 OK) o un mensaje de error si falla la validación (400 Bad Request).
- obtenerHistorialEntrenamientos(Long usuarioid): ResponseEntity<List>
 - **Descripción:** Obtiene el historial completo de entrenamientos de un usuario ordenado de forma descendente por fecha.
 - **Ruta:** /api/entrenamientos/usuario/{usuarioid} (método GET)
 - **Parámetros:** * usuarioid: ID del usuario a consultar.
 - **Respuesta:** Devuelve una lista JSON limpia con las sesiones del usuario (200 OK).
- obtenerDetalleEntrenamiento(Long id): ResponseEntity
 - **Descripción:** Muestra los detalles completos (ejercicios, series, equipamiento) de una sesión de entrenamiento específica.
 - **Ruta:** /api/entrenamientos/{id} (método GET)
 - **Parámetros:** * id: Identificador del entrenamiento.
 - **Respuesta:** Devuelve los datos de la sesión (200 OK) o un estado no encontrado si no existe (404 Not Found).

Métodos Privados:

- `mapearAEntrenamientoResponse(Entrenamiento entidad)`
 - **Descripción:** Convierte y desglosa la jerarquía completa de la entidad Entrenamiento (junto a sus ejercicios y series) en DTOs para estructurar la respuesta JSON final.
 - **Parámetros:** * entidad: Objeto Entrenamiento recuperado de la base de datos.
 - **Retorno:** Devuelve el objeto EntrenamientoResponse construido con todas sus relaciones limpias.